// Power Platform & Fabric User Group Cologne · 8 Dec 2026

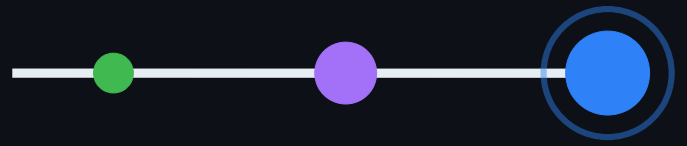
Stop Copy-Pasting Your Pipelines

Metadata-driven orchestration in practice



David Fengler

Azure Cloud Architect · glueckkanja AG





// whoami

David Fengler

Azure Cloud Architect @ glueckkanja AG

I build data foundations — lakehouses, pipelines, governance — and turn them into AI that holds up in production, not just in demos.

- blog datadave.dev
- github github.com/phant0mw0lf
- linkedin linkedin.com/in/davidfengler
- oss pipetree.dev

// agenda

What we'll cover

01 **The problem** ten systems, ten copies

02 **The model** YAML schema and lakehouse layout

03 **The executor** parallel runs, retries, failures, logging

04 **Design decisions** what worked, what didn't

05 **Live demo** a new source in Fabric, end to end

// 01

Ten systems, ten copies

How a data platform grows when copying is the fastest way forward



// 01 · the problem

It started with one good notebook

01

ERP first

Dynamics 365 via Export to Data Lake. Parameterised from day one — new instances never touched the file.

02

Then the rest

CRM via Synapse Link, SharePoint and Entra via Graph, on-prem SQL Server over JDBC. Copy, swap the reader, adjust.

03

Ten files

Each one generic within its system. Each one its own step in the job. The problem sat between the files.

// 01 · the problem

Then came the cross-cutting change

10

source systems, each with its own copied file

1

change: lineage columns everywhere, then multithreading

10×

the same edit, ten times, drifting a little each time

// 01 · the insight

What differed between the systems wasn’t logic. It was description.

which system · which object · which key · how to merge

// 02

Describe it, don’t write it

One YAML file, one small package that interprets it



// 02 · the model

Four things, kept apart

01

What

The metadata: tables, systems, keys, merge strategy.

02

In what order

A dependency graph the package derives itself.

03

How

An engine-agnostic executor. Spark is an adapter.

04

Where

Versioned files in the repo, promoted through CI/CD.

// 02 · the model

One table, one block

pipeline.yaml

```yaml
systems:                        # declared once, referenced by name
  crm_dataverse:
    type: csv                    # csv · json · parquet · sqlserver · kusto · custom …
    path: data/customer.csv

bronze:
  tables:
    customer:
      source: { system: crm_dataverse, object: account }
      business_key: [accountid]
      strategy: scd2                # scd2 · scd1 · replace · append
```

// 02 · the model

Merge is the strategy, full stop

strategy

- scd2 — full history
- scd1 — latest state
- replace — rebuild every run
- append — add rows

merge knobs

- sequence_by — which change wins
- delete_when — what marks a delete
- delete_mode — soft · hard · ignore
- ignore_columns — not a real change

// 02 · the model

Dependencies come from the code

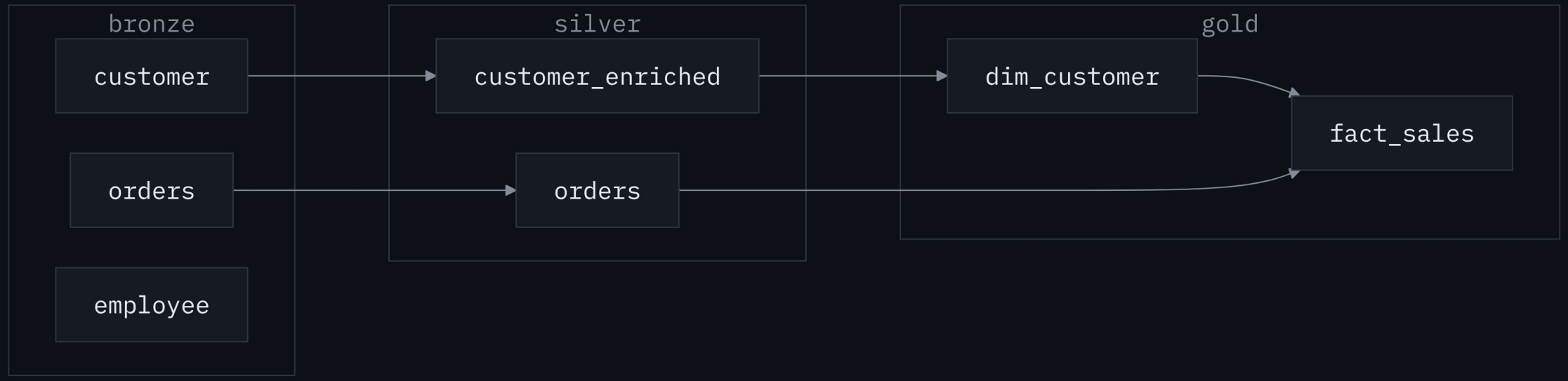
notebooks/gold/fact_sales.py

```
# one output table per file, inputs read at the top
orders   = spark.table("silver.orders")
customer = spark.table("gold.dim_customer")
# everything after — joins, logic — is yours

# pipeline.yaml
fact_sales:
  logic: notebooks/gold/fact_sales.py
  depends_on: auto        # parsed from the reads above
```

// 02 · the model

The package draws the tree



topological order · cycle check · run a table plus everything downstream

// 02 · the lakehouse

One lakehouse, a schema per layer

01

bronze · silver · gold

Schemas in one Fabric lakehouse, not one lakehouse per layer. Layers organise — they don't gate execution.

02

Files/

pipeline.yaml, the logic notebooks and landed source files. Versioned in the repo, deployed together.

03

_meta

pipeline_run_log: one row per table and run. Audit columns on every row, keyed to the same run.

// 02 · the lakehouse

Audit columns you never write again

every table

- _inserted_at · _updated_at
- _is_deleted
- _execution_id
- _source_system

scd2 tables, additionally

- _valid_from · _valid_to
- _is_current
- timestamps: UTC bigint, YYMMddHHmmssSSS
- readable in a debug query, lines up with the run

// 03

Run the tree, not the layers

Parallel, with retries, and failures that stop only what depends on them



// 03 · the executor

One counter per table

pipetree/executor/runner.py (trimmed)

```
pending = {t: len(parents[t]) for t in run_set}
submit(t for t, n in pending.items() if n == 0)

while futures:
    done = wait(futures, return_when=FIRST_COMPLETED)
    for table in done:
        for child in children[table]:
            pending[child] -= 1
            if pending[child] == 0: submit(child)
```

// 03 · the executor

Retry what can be retried

transient → retry

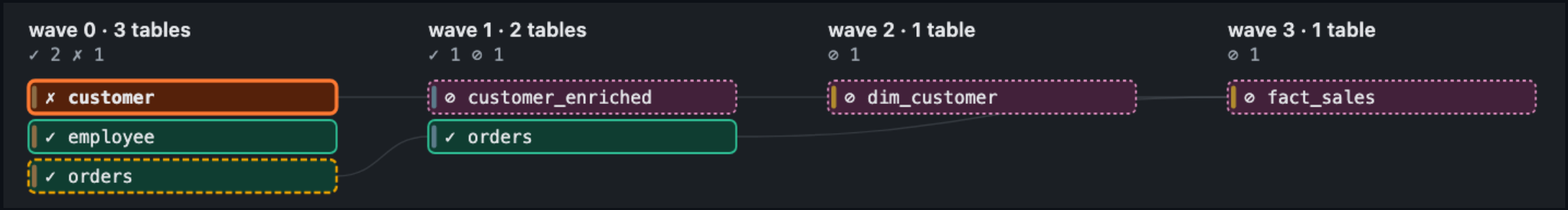
- Throttling, timeouts, dropped connections
- Spark/Delta errors translated by the adapter (0.1.1)
- Up to 3 attempts, full-jitter backoff

permanent → fail now

- Malformed export, missing column
- Waiting never fixes these
- append retries only if the run’s rows can be deleted first

// 03 · the executor

A failure stops exactly its subtree



failed ≠ upstream_failed — one error to read, not seven

// 03 · the executor

The digest, and an honest exit code

run output

```
bronze.orders: attempt 1 failed (Throttled: 429), retrying in 0.47s
bronze.orders: retried→succeeded (attempts=3)

Run 261015095124804: FAILED (exit code 1)
  failed              bronze.customer
  succeeded           bronze.employee
  retried→succeeded   bronze.orders
  upstream_failed     silver.customer_enriched
  upstream_failed     gold.dim_customer
  upstream_failed     gold.fact_sales
  succeeded           silver.orders
```

// 03 · logging

Every run leaves a trail

01

Run log

_meta.pipeline_run_log: status, attempts, duration, error per table. Written once per run, idempotent on _execution_id.

02

Live graph

LiveGraphView redraws the tree in the notebook while tables run. HtmlFileObserver does it anywhere else.

03

Alerts

A failed run can send one email via Azure Communication Services — token credential, no connection string.

// 04

What worked, what didn’t

And what I’d change today



// 04 · design decisions

Looking back

01

Worked

depends_on: auto. Tree over layers. failed vs upstream_failed. No locks in the scheduler.

02

Hurt

The simple parser misses dynamic table names. Engine errors needed translating before retries worked. append is hard to retry.

03

Today I’d …

TODO David: what you’d change today

// 05

A new source, live

products.parquet → bronze → gold.fact_sales → Power BI



$ live demo_

%pip install pipetree-meta · Fabric notebook · Direct Lake report

// 05 · recap

Everything the new source took

pipeline.yaml (+ one join in fact_sales.py)

```yaml
systems:
  shop_products:
    type: parquet
    path: data/products.parquet

bronze:
  tables:
    products:
      source: { system: shop_products, object: products }
      business_key: [product_id]
      strategy: scd1
```

// 05 · recap

One new node, one new edge



// takeaway

A new source is a config change, not a copy.

describe it once · let the package run it

// try it

Take it home

01

Install it

pip install pipetree-meta — docs and quick start at pipetree.dev. In Fabric: one %pip cell in the notebook, an environment for scheduled runs.

02

The long version

The “Pipeline Engineering” series on datadave.dev — schema, executor, Fabric, Databricks.

03

Slides & demo

PDF, pipeline.yaml, sample data and the notebook on the talk page.

// thanks

Thank you.

Questions, war stories, pushback — all welcome.

- blog datadave.dev
- github github.com/phant0mw0lf
- linkedin linkedin.com/in/davidfengler
- oss pipetree.dev



datadave.dev/talks